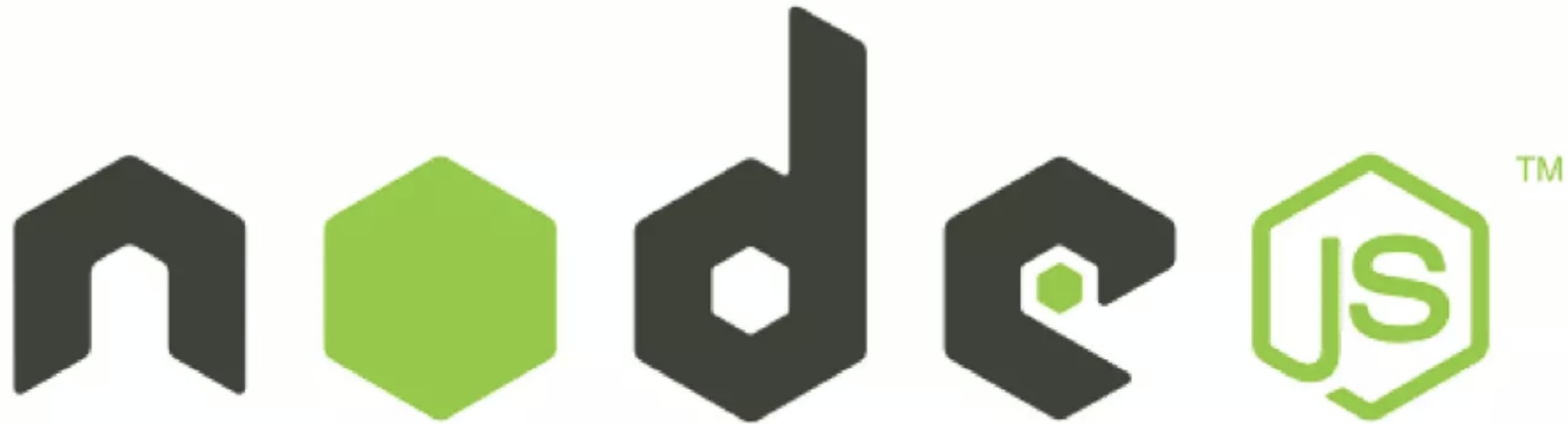


24CSA212 – Full Stack Frameworks

INTRODUCTION TO NODE.JS AND NPM



Basic Concept of Node.Js & NPM

WHAT IS NODEJS?

- NodeJS is an open source, cross platform runtime environment for server side and networking application.
- Basically, NodeJS is server side programming language like PHP, C#, Python etc.
- In 'Node.js' , '.js' doesn't mean that its solely written JavaScript. It is 40% JS and 60% C++.
- NodeJS is cross platform. So it runs well on Linux systems, Mac, can also run on Windows systems.
- It provided an event driven architecture and a non blocking I/O that optimize and scalability. These technology uses for real time application

Key Features of Node JS

Fast Data Streaming

Single - Threaded

Highly Scalable

Event - Driven

JavaScript

No Buffering

Cross - Platform Compatibility

Asynchronous



Fast Data Streaming

Node.js uses streams to process data in small chunks instead of loading everything into memory, making it very fast for handling large files and real-time data.

Single-Threaded

Node.js runs on a single main thread, but it can handle many requests efficiently using non-blocking operations instead of creating multiple threads.

Single-threaded: Unlike traditional server environments that create a new thread for each request, Node.js runs in a single process. This eliminates the overhead of managing multiple threads and reduces the chances of bugs related to thread concurrency.

Highly Scalable

Because of its event-driven and non-blocking nature, Node.js can handle thousands of simultaneous client requests with low resource usage.

Event-Driven

Node.js works by responding to events (like requests, file reads, or database calls) using callbacks, promises, or async/await.

Key Features of Node JS

Fast Data Streaming

Single - Threaded

Highly Scalable

Event - Driven

JavaScript

No Buffering

Cross - Platform Compatibility

Asynchronous



JavaScript

Node.js allows JavaScript to be used on the server side, enabling developers to use the same language for both frontend and backend development.

No Buffering

Node.js processes data as it arrives instead of waiting for the complete data to load, which improves performance and reduces memory usage.

Cross-Platform Compatibility

Node.js applications can run on multiple operating systems like Windows, Linux, and macOS without code changes.

Asynchronous

Node.js performs time-consuming tasks (like I/O operations) asynchronously, allowing other operations to continue without waiting.

Asynchronous and Non-blocking: Node.js uses asynchronous programming, which means it can handle multiple operations simultaneously without waiting for any operation to complete. This makes Node.js highly efficient and capable of handling thousands of concurrent connections with a single server.

Rich Standard Library: Node.js comes with a comprehensive standard library that includes modules for handling file systems, networking, and other essential tasks. This makes it easier to build robust applications without relying on external libraries.

WHAT IS NODEJS?

- It used Google JavaScript V8 Engine to Execute Code.
- **V8** is an open source JavaScript engine developed by Google. Its written in C++ and is used in Google Chrome Browser.
- NodeJS invented in 2009 by Ryan Dahl.
- NodeJS is popular in development because front & backend side both uses JavaScript Code.

‘Node's goal is to provide an easy way to build scalable network programs’ - (from nodejs.org!)

REASONS TO USE NODEJS

- It is Open Source and cross platform. It runs well on Linux systems, Mac, can also run on Windows systems.
- It is fast.
- Node.js wins with Ubiquity the fact that something or someone seems to be everywhere:
- Node.js wins with Data Streaming
- Node.js wins with Database Queries
- Real Time web applications Support
- Node.js –Effective Tooling with NPM
- Many more...

Why Learn Node.js

- It Enables the use of JavaScript for both frontend and backend development.
- It Supports building real-time applications like chat apps and gaming servers.
- It Provides high scalability for I/O-heavy applications.
- It Backed by a vibrant community and extensive library support

CONFUSING WITH NODEJS

- Programs for Node.js are written in JavaScript but not in the same JavaScript we are use to. There is no DOM implementation provided by Node.js, i.e. you **can not** do this:

```
var element = document.getElementById("elementId");
```

- Everything inside Node.js runs in a **single-thread**.

WHO USES NODEJS?

Node.js users span a variety of industries

News Corp

Angie's list

Zappos

CBS

Symantec

NORDSTROM

Johnson
Controls

IBM

coursera

change.org

RBS

flickr

Koding

PayPal

LOWE'S

Google

NETFLIX

DOW JONES

amazon

SONY

redhat

fitbit

Intel

Aol.

SAP

NodePing

Pinterest

Joyent

intuit

GoPro

SIEMENS

Medium

QUALCOMM

LogMeIn

CONDÉ NAST

BUSTLE

LendingClub

Fidelity

Microsoft

IBM

IBM

IBM

IBM

IBM

IBM

IBM

IBM

IBM

IBM

IBM

IBM

IBM

UBER

LinkedIn

LinkedIn

The New York Times

ADP

ebay

WIX

Quizlet

TARGET

Intel

Cigna

MasterCard

dispego.com

Capital One

Walmart

GROUPON

GoDaddy

NASA

airbnb

citi

SAMSUNG

Trello

YAHOO!

EA

Quad

asana

Mozilla

NEW YORKER

Twitter

Microsoft

CISCO

Yandex

DuckDuckGo

SKYCATCH

Y&Yammer

OpenTable

pwc

reach

shutterstock

node

WHO IS USING NODE.JS IN PRODUCTION?

- **Yahoo!** : iPad App **Livestand** uses Yahoo! Manhattan framework which is based on Node.js.
- **LinkedIn** : LinkedIn uses a combination of Node.js and MongoDB for its mobile platform. iOS and Android apps are based on it.
- **eBay** : Uses Node.js along with ql.io to help application developers in improving eBay's end user experience.
- **Dow Jones** : The WSJ Social front-end is written completely in Node.js, using Express.js, and many other modules.
- Complete list can be found at:
<https://github.com/joyent/node/wiki/Projects,-Applications,-and-Companies-Using-Node>

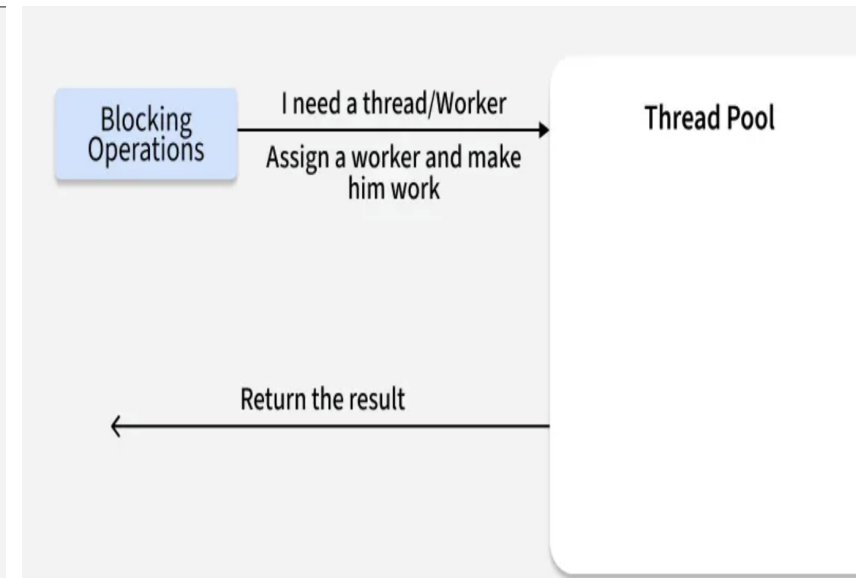
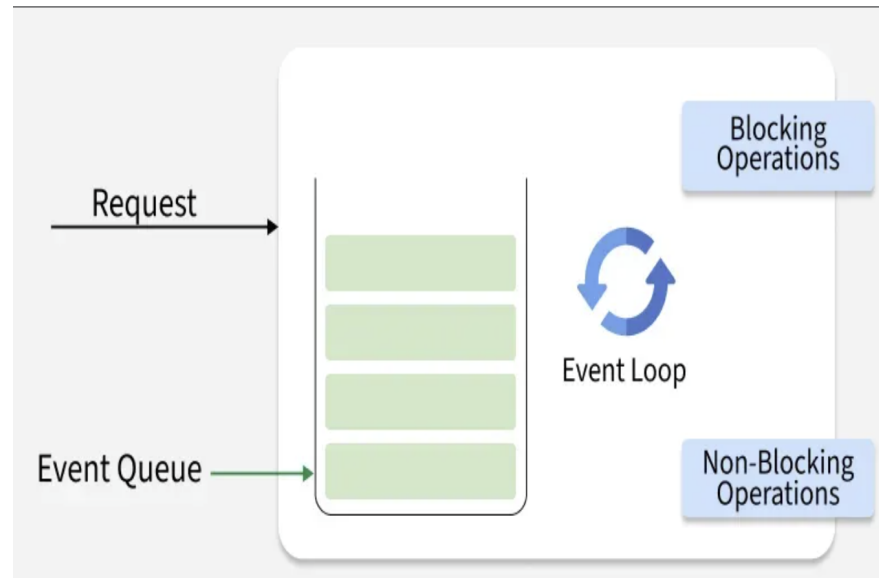
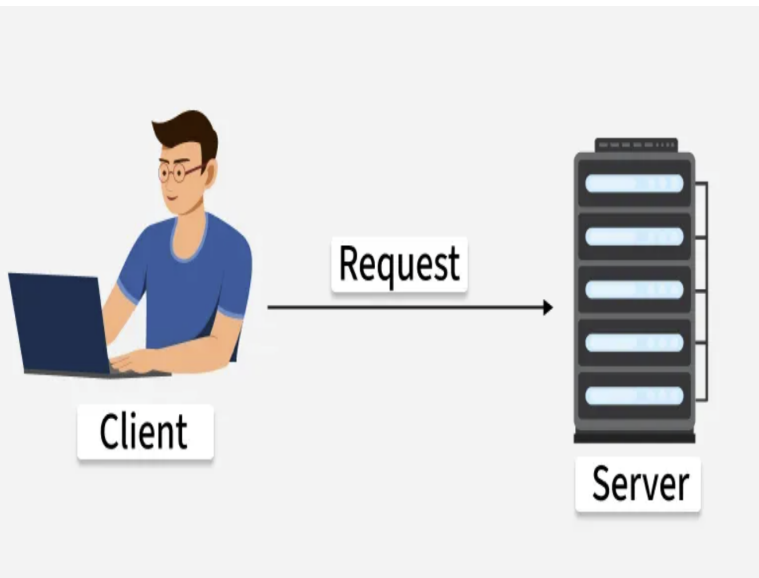
Working of Node.js

Node.js is a runtime environment that enables JavaScript to run outside the browser for building scalable server-side applications.

Built on the V8 JavaScript engine.

Uses asynchronous, event-driven architecture.

Suitable for scalable network applications.



Single-Threaded Event Loop Model

Node.js operates on a single thread but efficiently handles multiple concurrent requests using an event loop.

Client Sends Request: Requests data, files, or database operations.

Event Loop Handles It: Non-blocking tasks are offloaded without stopping execution.

Background Processing: Node.js continues handling other requests.

Callback Executes: Response is returned once the task completes.

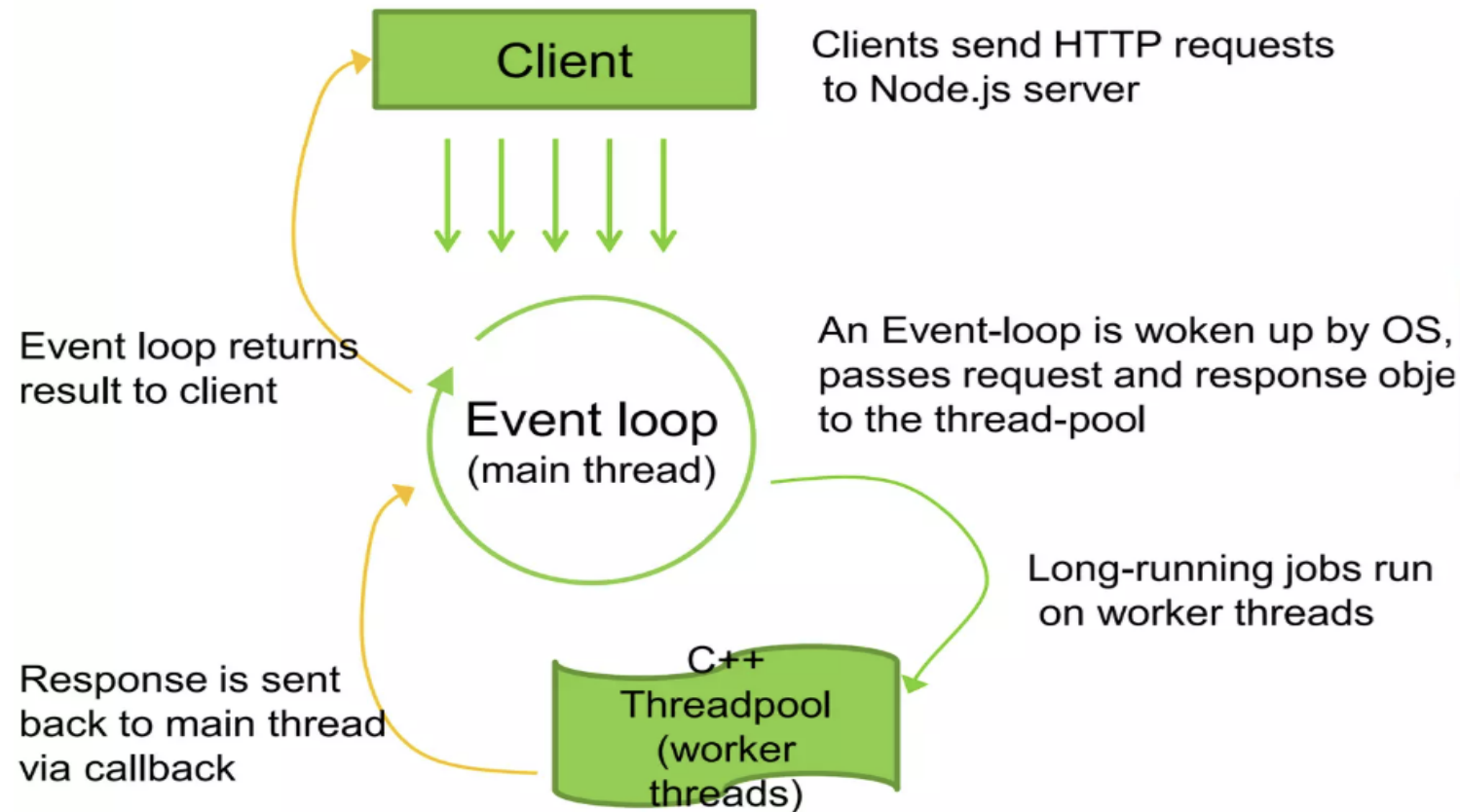
```
const fs = require('fs');

fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});

console.log("Reading file...");
```


NODEJS EVENT LOOP

- Event-loops are the core of event-driven programming, almost all the UI programs use event-loops to track the user event, for example: Clicks, Ajax Requests etc.



JavaScript		C/C++		
node standard library				
node bindings (socket, http, etc)				
V8	thread pool (libeio)	event loop (libev)	DNS (c-ares)	crypto (OpenSSL)

INTRODUCTION TO NPM (NODE PACKAGE MANAGER)

- npm is the package manager for Node.js and JavaScript
- We can create public & private module
- Find all modules on NPM website : <https://www.npmjs.com/>
- There 5.5 Lacks+ packages available So in world its largest JavaScript package manager.
- Below are some most popular packages which used in many projects.

- lodash
- express
- request

- react
- debug
- bluebird

- async
- chalk
- commander

SOME GOOD MODULES

- **Express** – to make things simpler e.g. syntax, DB connections.
- **Jade** – HTML template system
- **Socket.IO** – to create real-time apps
- **Nodemon** – to monitor Node.js and push change automatically
- **CoffeeScript** – for easier JavaScript development

A package contains all the files needed for a module and modules are the JavaScript libraries that can be included in Node project according to the requirement of the project. NPM can install all the dependencies of a project through the [package.json](#) file.

It can also update and uninstall packages.

In the [package.json](#) file, each dependency can specify a range of valid versions using the semantic versioning scheme, allowing developers to auto-update their packages while at the same time avoiding unwanted breaking changes.

Packages

npm manages downloads of dependencies of your project.

Installing all dependencies

If a project has a package.json file, by running **npm install** it will install everything the project needs, in the node_modules folder, creating it if it's not existing already.

Installing a single package

`npm install <package-name>`

since npm 5, this command adds `<package-name>` to the `package.json` file dependencies. Before version 5, you needed to add the flag `--save`.

`--save-dev` installs and adds the entry to the `package.json` file `devDependencies`

`--no-save` installs but does not add the entry to the `package.json` file dependencies

`--save-optional` installs and adds the entry to the `package.json` file `optionalDependencies`

`--no-optional` will prevent optional dependencies from being installed

Shorthands of the flags can also be used:

`-S: --save`

`-D: --save-dev`

`-O: --save-optional`

The difference between `devDependencies` and `dependencies` is that the former contains development tools, like a testing library, while the latter is bundled with the app in production.

Updating packages:

npm update

npm will check all packages for a newer version that satisfies your versioning constraints.

single package to update:

```
npm update <package-name>
```

Creating a Node Project:

To create a Node project, **npm init** is used in the folder in which user want to create project.

The npm command line will ask a number of questions like **name, license, scripts, description, author, keywords, version, main file** etc.

After npm is done creating the project, a package.json file will be visible in project folder as a proof that the project has been initialized.

Node.js package.json

The **package.json** file is the heart of Node.js system. It is the manifest file of any Node.js project and contains the metadata of the project.

The package.json file contains the metadata information. This metadata information in **package.json** file can be categorized into below categories.

- 1. Identifying metadata properties:** It basically consist of the properties to identify the module/project such as the name of the project, current version of the module, license, author of the project, description about the project etc.
- 2. Functional metadata properties:** As the name suggests, it consists of the functional values/properties of the project/module such as the entry/starting point of the module, dependencies in project, scripts being used, repository links of Node project etc.

Create a package.json file: A **package.json** file can be created in two ways.

1. Using npm init
 2. Writing directly to file
-

NODEJS INSTALLATION

- Download at <https://nodejs.org/en/download/>
- Available in installable file and source code.
- It is easy to install so you can start start developing today on any platform.
- In latest version npm (Node Package Manager) already available inside it so no need to download npm separately.

COMMAND LINE INTRO & NODE COMMAND

- Windows platform, Open command prompt (cmd)
- Below are some node basic command introduction
 - `node` : open node editor and write and execute node JavaScript code there.
 - `node -v` : check the node version
 - `node -h` : help command
 - `node [packagename]`: it runs your package (module)

NODE.JS ECOSYSTEM

- Node.js heavily relies on **modules**, in previous examples **require** keyword loaded the http modules.
- Creating a module is easy, just put your JavaScript code in a separate js file and include it in your code by using keyword require, like:

```
var modulex = require('./modulex');
```

- Libraries in Node.js are called packages and they can be installed by typing

```
npm install "package_name"; //package should be  
available in npm registry @ nmpjs.org
```

- **NPM** (Node Package Manager) comes bundled with Node.js installation.

Advantages of Node.js

- **Performance:** Node.js is built on the V8 engine, which compiles JavaScript to native machine code, making it extremely fast.
- **Scalability:** Node.js can handle a large number of simultaneous connections with high throughput, making it suitable for scalable network applications.
- **Unified Language:** Using JavaScript on both the client and server sides simplifies development and allows for code reuse.

Node.js is a powerful tool for building efficient and scalable applications. Its asynchronous, non-blocking nature, combined with the ability to use JavaScript on the server, makes it a popular choice among developers.

Example: Creating a Simple HTTP Server

Here is a simple example of creating an HTTP server using Node.js:

```
// Import the http module
const { createServer } = require('node:http');

// Define the hostname and port
const hostname = '127.0.0.1';
const port = 3000;

// Create the server
const server = createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World\n');
});

// Start the server and listen on the specified port and hostname
server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

To run this code, save it as `server.js` and execute it using the command `node server.js` in your terminal.

It will start a server,
and when you visit
`http://localhost:3000`,
it will display

Hello World

- The http module is imported to create a basic HTTP server.
- The `createServer()` method is used to handle incoming requests and send responses.
- The server listens on port 3000, and a message is displayed in the browser when accessed.

To run this snippet, save it as a `server.js` file and run `node server.js` in your terminal.

This code first includes the Node.js `http` module.

Node.js has a fantastic standard library, including first-class support for networking.

The `createServer()` method of `http` creates a new HTTP server and returns it.

The server is set to listen on the specified port and host name. When the server is ready, the callback function is called, in this case informing us that the server is running.

Whenever a new request is received, the request event is called, providing two objects: a request (an `http.IncomingMessage` object) and a response (an `http.ServerResponse` object).

Those 2 objects are essential to handle the HTTP call.

The first provides the request details. In this simple example, this is not used, but you could access the request headers and request data.

The second is used to return data to the caller.

In this case with:

```
res.statusCode = 200;
```

we set the statusCode property to 200, to indicate a successful response.

We set the Content-Type header:

```
res.setHeader('Content-Type', 'text/plain');
```

and we close the response, adding the content as an argument to end():

```
res.end('Hello World\n');
```

How Node works?

Node accepts the request from the clients and sends the response, while working with the request node.js handles them with a single thread.

To operate I/O operations or requests node.js use the concept of threads.

Thread is a sequence of instructions that the server needs to perform. It runs parallel on the server to provide the information to multiple clients.

Node.js is an event loop single-threaded language. It can handle concurrent requests with a single thread without blocking it for one request.

Components of Node.js Architecture

Node.js architecture consists of components that work together to handle asynchronous, non-blocking operations efficiently.

V8 Engine: Compiles JavaScript to machine code for fast execution.

Event Loop: Manages asynchronous tasks without blocking the main thread.

Libuv: Handles I/O operations, thread pool, and timers.

Non-Blocking I/O: Executes tasks without waiting for previous ones to complete.

Use Cases of Node.js

Node.js is well suited for building high-performance, scalable, and real-time applications. Common use cases include:

Web APIs and Backend Services: Used to build RESTful APIs and GraphQL APIs and power backend services for web and mobile applications.

Real-Time Applications: Chat applications (e.g., WhatsApp, Slack). Live streaming services (e.g., Netflix, Twitch).

Microservices Architecture: Enables building scalable, independent services, commonly used in cloud-based applications.

IoT (Internet of Things) Applications: Handles real-time data streams from devices, making it suitable for smart homes and sensor-based systems.

Serverless Computing: Efficiently runs lightweight functions on cloud platforms like AWS Lambda and Azure Functions.

Single-Page Applications (SPAs): Supports React, Angular, and Vue.js apps by efficiently handling backend API requests.

Data-Intensive Applications: Handles real-time analytics and integrates well with NoSQL databases like MongoDB and Firebase.

Differences between Node.js and the Browser

Both the browser and Node.js use JavaScript as their programming language. Building apps that run in the browser is a completely different thing than building a Node.js application. Despite the fact that it's always JavaScript, there are some key differences that make the experience radically different.

From the perspective of a frontend developer who extensively uses JavaScript, Node.js apps bring with them a huge advantage: the comfort of programming everything - the frontend and the backend - in a single language.

In the browser, most of the time what you are doing is interacting with the DOM, or other Web Platform APIs like Cookies. Those do not exist in Node.js, of course. You don't have the document, window and all the other objects that are provided by the browser.

And in the browser, we don't have all the nice APIs that Node.js provides through its modules, like the filesystem access functionality.

We can use both `require()` and `import` in Node.js, while you are limited to `import` in the browser.

Another difference is that Node.js supports both the CommonJS and ES module* systems (since Node.js v12), while in the browser we are starting to see the ES Modules standard being implemented.

In Node.js you control the environment. Unless you are building an open source application that anyone can deploy anywhere, you know which version of Node.js you will run the application on. Compared to the browser environment, where you don't get the luxury to choose what browser your visitors will use, this is very convenient.

The ES module format is the official standard format to package JavaScript code for reuse, and most modern web browsers natively support the modules.

